

ADLC tra letteratura esistente e proposta originale

Sintesi della risposta

La conclusione più solida, alla luce delle fonti accademiche e industriali oggi reperibili, è questa: **il tuo ADLC, inteso come framework repository-based di AI-Driven Software Development Life Cycle con file persistenti di memoria, vincoli operativi e continuità degli agenti, non risulta ancora come standard accademico consolidato con questa definizione precisa.** Tuttavia, **non nasce nel vuoto:** si appoggia in modo molto credibile a filoni già robusti e riconoscibili, fra cui **agentic AI** nello SDLC, artefatti di contesto persistente, prompt engineering e planning before action, human-in-the-loop, governance del rischio, DevSecOps e ADR. Inoltre, la sigla **ADLC** è già usata in modo esplicito in ambito enterprise per **Agent Development Lifecycle** da IBM, Microsoft e Salesforce, mentre AWS usa la nomenclatura adiacente **AI-DLC** per un lifecycle AI-driven dello sviluppo software. ¹

Per questo, la formula più onesta e più forte non è “ADLC è già una metodologia consolidata in letteratura” ma piuttosto: **“ADLC è un framework pratico proposto in questo libro, che sintetizza e rende operativi filoni già consolidati.”** È una posizione metodologicamente difendibile proprio perché le componenti fondative esistono già, mentre l'assemblaggio specifico che proponi appare ancora come una **specializzazione originale** del tema. ²

Dove si colloca oggi il termine ADLC

Nel lessico enterprise contemporaneo, **ADLC** è ormai usato in modo esplicito per **Agent Development Lifecycle**. IBM lo definisce come una metodologia end-to-end per costruire e gestire agenti AI in modo affidabile, interoperabile e governato; inoltre distingue l'ADLC dallo SDLC classico perché gli agenti falliscono in modo diverso dal software deterministico, per esempio tramite allucinazioni, problemi di allineamento, violazioni di vincoli o output plausibili ma falsi. Per questo IBM mette al centro osservabilità, containment, valutazione continua, sandboxing, versioning, rollback e monitoraggio. ³

Anche Microsoft Foundry usa il termine **agent development lifecycle** per un percorso che va dalla creazione iniziale al monitoraggio in produzione, con enfasi su versioning immutabile, tracing, valutazioni ripetibili di qualità e safety, monitoraggio post-pubblicazione e controllo stretto di identità e permessi. Questo rafforza l'idea che, nel 2026, **ADLC non è più un'etichetta isolata**, ma un campo terminologico già attivo nell'industria. ⁴

Salesforce converge nella stessa direzione: presenta l'ADLC come un framework specifico per agenti non deterministici, con fasi di ideazione e design, sviluppo, testing e validazione, deployment e monitoraggio/tuning continuo, mappando esplicitamente il rapporto fra SDLC tradizionale e ciclo di vita degli agenti. ⁵

In parallelo, AWS usa una sigla vicina ma distinta, **AI-DLC**, per una metodologia di sviluppo software AI-driven in cui l'AI crea piani, cerca chiarimenti, implementa dopo validazione umana e **mantiene contesto persistente nel repository** attraverso piani, requisiti e artefatti di design. Questa letteratura industriale è particolarmente vicina al tuo impianto, anche se usa un'etichetta diversa. ⁶

C'è poi un secondo problema terminologico: in parte della letteratura professionale sul software, **ADLC** continua a significare **Application Development Life Cycle**, spesso quasi come sinonimo operativo di SDLC. In altre parole, la sigla è già "occupata" da almeno due tradizioni: una storica, legata all'application development, e una nuova, legata all'agent development. Questo rende consigliabile una **nota terminologica esplicita** nel libro. ⁷

I filoni accademici che sostengono davvero il tuo framework

Sul piano accademico, il mattone più importante è la letteratura su **agentic AI nello sviluppo software**. Una systematic literature review del 2026, costruita su 92 studi primari verificati manualmente, conclude che la **verificabilità degli output** è il principale fattore che abilita l'adozione degli agenti nello SDLC; mostra inoltre che le fasi tarde del lifecycle, quelle con feedback eseguibile e controllabile, sono oggi più mature delle fasi iniziali. La stessa review identifica come pattern dominante la specializzazione **Planner-Executor-Reviewer**, con il Reviewer che realizza la verificabilità tramite loop di feedback eseguibili, e osserva che l'industria tende a confinare gli agenti in spazi d'azione **verificabili e delimitati**. Questo è molto vicino alla tua idea di fasi, checkpoint, confidence, halt trigger e controllo dell'azione. ⁸

Un secondo contributo recente sul cosiddetto **Agentic SDLC** sostiene che l'oggetto centrale non è più solo la generazione di codice, ma la **delegated execution under human supervision**. Lo studio mette a fuoco cinque problemi aperti — valutazione, governance, technical debt, redistribuzione delle competenze ed economia dell'attenzione — che coincidono in larga parte con i rischi che il tuo framework cerca di gestire tramite memoria persistente, vincoli procedurali e artifacts nel repository. ⁹

Il filone del **planning before action** è anch'esso ben fondato. Il paper di Wei et al. mostra che la generazione di passaggi intermedi di ragionamento, cioè chain-of-thought, **migliora significativamente** la capacità dei modelli di affrontare problemi complessi. Il paper ReAct estende poi il quadro combinando reasoning e acting in traiettorie interallacciate, in cui il modello aggiorna piani, gestisce eccezioni e interagisce con fonti esterne. Nel tuo ADLC, l'idea che l'agente debba pianificare, rendere espliciti i passaggi e agire entro spazi controllati è dunque pienamente allineata con letteratura già esistente, anche se non specificamente etichettata col tuo nome. ¹⁰

La conseguenza importante è che **non devi dimostrare che esista già "il tuo ADLC" come scuola consolidata**. Ti basta mostrare che il tuo framework è una **sintesi operativa** di filoni riconosciuti: lifecycle engineering, agentic software development, reasoning/planning, human oversight e controlli di verificabilità. È esattamente qui che la tua proposta guadagna forza. ¹¹

Il ruolo degli artifact di contesto persistente

La parte più distintiva del tuo ADLC — file nel repository che danno memoria, regole, continuità e auditabilità agli agenti — non è fantasia isolata: è **già diventata pratica prodotta**. OpenAI documenta `AGENTS.md` come un meccanismo di istruzioni persistenti che Codex legge prima di iniziare il lavoro, con regole di scoperta lungo la gerarchia del repository, ordine di precedenza e limiti di dimensione. Anthropic documenta `CLAUDE.md` come memoria di progetto caricata all'inizio di ogni sessione, con scope organizzativo, utente, progetto e locale, e raccomanda di mantenerla concisa, specifica, coerente e periodicamente ripulita da istruzioni obsolete o confliggenti. Cursor documenta a sua volta istruzioni persistenti tramite Project, Team e User Rules, oltre al supporto per `AGENTS.md`. AWS, sul versante AI-DLC, descrive esplicitamente il salvataggio e la manutenzione di **persistent context across all phases** nel repository. ¹²

Anthropic spinge il parallelismo ancora più vicino al tuo modello: `CLAUDE.md` può contenere comandi di build, convenzioni, layout del progetto, decisioni architetturali, naming conventions e workflow condivisi; inoltre Claude Code suggerisce di spostare procedure complesse o path-specific in meccanismi più mirati, e dichiara esplicitamente che file troppo lunghi consumano contesto e **riducono l'aderenza** del modello. Questa è quasi una formalizzazione industriale del problema che, nel tuo linguaggio, chiameresti memoria utile, vincoli stabili e continuità governata. ¹³

Sul piano della ricerca, il concetto di **context rot** è ormai esplicitato in letteratura software: un paper del 2026 definisce il problema come l'invecchiamento del contesto persistente fornito agli assistenti di coding attraverso file come `CLAUDE.md`, `AGENTS.md` e `.cursorrules`, mostrando che questi artefatti possono divergere dal repository man mano che il software evolve. Gli autori collegano il problema a decenni di ricerca sulla consistenza fra documentazione e codice e propongono di riusare strumenti di documentation consistency per rilevare il rot del contesto. ¹⁴

La letteratura empirica più recente è interessante perché **non è tutta unidirezionale**. Da un lato, uno studio su `AGENTS.md` conclude che i context files non migliorano in generale il task success rate e aumentano il costo inferenziale di oltre il 20%; aggiunge però che gli agenti seguono davvero le istruzioni presenti nei file, e che i repository overviews molto generici non risultano particolarmente utili. Dall'altro lato, uno studio diverso trova che la presenza di `AGENTS.md` è associata a **runtime mediani più bassi** e a **minore consumo di output tokens**, con comportamento di completamento comparabile. Tradotto per il tuo libro: il valore dei file di contesto non è "magico", ma dipende da **qualità, minimalità, aggiornamento, specificità e integrazione nel flusso di verifica**. ¹⁵

Questa è una notizia positiva per il tuo impianto. Significa che `_CONTEXT.md`, `PROGRESS.md`, regole di arresto, confidence tag e file JSON di stato possono essere difesi non come folklore, ma come **design choices da trattare con disciplina ingegneristica**, proprio perché la ricerca mostra che contesto mal progettato o non aggiornato può introdurre costo, dispersione e falsi allineamenti. ¹⁶

Governance, rischio e memoria decisionale

Il tuo ADLC regge ancora meglio quando viene collegato ai filoni di **governance e risk control**. Il profilo NIST per la Generative AI afferma che i sistemi GAI possono richiedere livelli diversi di **human oversight**, configurazioni diverse di human-AI teaming, e spesso anche **additional human review, tracking, documentation, and greater management oversight**. NIST collega inoltre la governance di questi sistemi a auditing, assessment, change management, incident response, monitoring, risk-based controls e secure software development practices. Tutto questo rafforza l'idea che un lifecycle per agenti non possa limitarsi a "prompt e codice", ma debba prevedere ruoli, logiche di stop, documentazione e accountability. ¹⁷

Dal punto di vista della sicurezza applicata, OWASP segnala fra i rischi principali per le applicazioni con LLM il **prompt injection**, l'**insecure output handling**, le **supply-chain vulnerabilities**, l'**excessive agency** e l'**overreliance**. Questi rischi sono precisamente il motivo per cui nel tuo framework hanno senso guardrail di repository, approvazioni umane, limiti d'azione, confidence tag e trigger di halt. ¹⁸

Sul lato DevSecOps, NIST ricorda che pochi modelli SDLC affrontano la sicurezza in modo esplicito e che pratiche come il **Secure Software Development Framework** devono essere integrate in ogni implementazione del lifecycle. In parallelo, IBM descrive il proprio ADLC come una disciplina **basata su pratiche standard DevSecOps**, pensata per mantenere gli agenti sicuri, affidabili e allineati ai vincoli organizzativi e regolatori. Dunque, se nel libro vuoi includere DevSecOps come componente fondativa del tuo ADLC, la scelta non è ornamentale: è metodologicamente supportata. ¹⁹

Infine, la parte di **ADR e decision logging** è perfettamente coerente con un ADLC repository-based. Martin Fowler definisce gli ADR come documenti brevi che catturano una decisione, il contesto e le conseguenze significative, e raccomanda di conservarli nel repository del prodotto, in markup leggero come Markdown, così che siano leggibili e diffabili insieme al codice. Microsoft aggiunge che un ADR dovrebbe essere un log append-only, con opzioni considerate, trade-off, stato e anche il **confidence level** della decisione. Questa corrispondenza con i tuoi artifact è quasi diretta. ²⁰

Posizionamento editoriale consigliato

La formulazione che ti consiglierei di adottare nel libro è questa: **ADLC è una proposta originale di framework operativo, non un'etichetta accademica già stabilizzata nella forma specifica che usi tu**. Conviene quindi presentarlo come una **specializzazione pratica** di filoni esistenti, non come una metodologia già codificata in letteratura con la tua stessa semantica. Questa prudenza è anche epistemicamente corretta, perché le fonti più vicine all'argomento sono per larga parte documentazione vendor e lavori molto recenti del 2026, inclusi preprint e workshop: segno di un'area emergente, non di un campo ormai canonizzato. ²¹

Una formulazione pronta da usare potrebbe essere questa:

ADLC, in questo libro, indica un framework pratico di AI-Driven Software Development Life Cycle. Non è presentato come uno standard accademico già consolidato nella sua forma specifica, ma come una sintesi originale e operativa di filoni maturi: software lifecycle engineering, agentic AI, prompt engineering e planning, contesto persistente nel repository, human-in-the-loop, governance del rischio, DevSecOps e architecture decision records. Questa frase è forte perché non sovravende il termine, ma lo ancora a basi già documentate. ²²

A livello terminologico, aggiungerei poi una nota del tipo: *La sigla ADLC è già usata in ambito enterprise soprattutto come Agent Development Lifecycle; in altre tradizioni professionali è stata usata anche per Application Development Life Cycle. In questo libro il termine viene ulteriormente specializzato per descrivere un lifecycle AI-driven dello sviluppo software, centrato su artifact persistenti nel repository, continuità del contesto e checkpoints di controllo umano. In questo modo non sembri ignorare l'uso IBM/Microsoft/Salesforce del termine, né la collisione con l'uso tradizionale.* ²³

Se vuoi un posizionamento ancora più preciso, potresti persino trattare il tuo ADLC come **una declinazione software-specifica dell'Agent Development Lifecycle**, oppure come **una forma repository-governed di AI-driven SDLC**. Sarebbe una mossa concettualmente elegante: da un lato ti colleghi a una terminologia enterprise ormai esistente; dall'altro chiarisci che il tuo contributo non è l'invenzione del lifecycle degli agenti in generale, ma la sua **operazionalizzazione concreta per coding agents dentro un repository reale**. È probabilmente la combinazione più forte fra originalità e onestà intellettuale. ²⁴

In definitiva, la tua intuizione centrale è corretta: **ADLC, nel senso preciso che vuoi dargli, non è ancora "la letteratura"**. Ma proprio per questo può diventare **la tua contribuzione**: non come pretesa di standard già riconosciuto, bensì come framework originale, difendibile e ben fondato su letteratura e pratiche già esistenti. ²⁵

2 8 11 21 22 25 [2605.15245] Assistance to Autonomy: A Systematic Literature Review of Agentic AI across the Software Development Life Cycle
<https://arxiv.org/abs/2605.15245>

4 Agent development lifecycle - Microsoft Foundry | Microsoft Learn
<https://learn.microsoft.com/en-us/azure/foundry/agents/concepts/development-lifecycle>

5 The Agent Development Lifecycle: From Conception to Production | Agentforce | Fundamentals | Salesforce Developers
<https://architect.salesforce.com/docs/architect/fundamentals/guide/agent-development-lifecycle.html>

6 AI-Driven Development Life Cycle: Reimagining Software Engineering | AWS DevOps & Developer Productivity Blog
<https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/>

7 Application Development Life Cycle & Management Models
https://www.couchbase.com/blog/application-development-life-cycle/?utm_source=chatgpt.com

9 [2604.26275] Agentic AI in the Software Development Lifecycle: Architecture, Empirical Evidence, and the Reshaping of Software Engineering
<https://arxiv.org/abs/2604.26275>

10 [2201.11903] Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
<https://arxiv.org/abs/2201.11903>

12 Custom instructions with AGENTS.md – Codex | OpenAI Developers
<https://developers.openai.com/codex/guides/agents-md>

13 How Claude remembers your project - Claude Code Docs
<https://docs.anthropic.com/en/docs/claude-code/memory>

14 16 [2606.09090] Context Rot in AI-Assisted Software Development: Repurposing Documentation Consistency for AI Configuration Artifacts
<https://arxiv.org/abs/2606.09090>

15 [2602.11988] Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?
<https://arxiv.org/abs/2602.11988>

17 Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile
<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>

18 OWASP Top 10 for Large Language Model Applications | OWASP Foundation
<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

19 Secure Software Development Framework | CSRC
https://csrc.nist.gov/projects/ssdf?utm_source=chatgpt.com

20 Architecture Decision Record
<https://martinfowler.com/bliki/ArchitectureDecisionRecord.html>